🏍️ BikerOrNot — Android App Build Plan

Kotlin · Jetpack Compose · Supabase · 10-Phase Development Guide for Claude Code

---

How to Use This Document

This document is your master build plan for the BikerOrNot Android app. At the start of each Claude Code session, paste this document along with the phase-specific prompt at the bottom of each phase section. Build phases in order — each one depends on the last.

The Android app shares the same Supabase backend as the web app. No separate backend is needed.

Tech Stack

Layer	Technology	Purpose
Language	Kotlin	Primary language
UI Framework	Jetpack Compose	Declarative UI
Architecture	MVVM + Clean Architecture	Separation of concerns
Navigation	Navigation Compose	Screen routing
Dependency Injection	Hilt	DI container
Backend	Supabase (shared with web app)	DB, Auth, Storage, Realtime
Supabase SDK	supabase-kt (official Kotlin SDK)	Supabase client
Networking	Ktor (bundled with supabase-kt)	HTTP client
Image Loading	Coil 3	Async image loading + caching
Local Storage	DataStore Preferences	User session, settings
Async	Kotlin Coroutines + Flow	Async operations, reactive streams
State Management	ViewModel + StateFlow + UiState	Screen state
Local Cache (optional)	Room	Offline feed caching (Phase 10)
Push Notifications	Firebase Cloud Messaging (FCM)	Push delivery

Layer	Technology	Purpose
Image Compression	Compressor (or custom)	Pre-upload compression
Testing	JUnit 4 · Mockk · Turbine	Unit + Flow testing
Build	Gradle (Kotlin DSL) · Version Catalogs	Dependency management
Min SDK	API 26 (Android 8.0)	Covers ~95% of active devices
Target SDK	API 35	Latest stable

Project Structure (Clean Architecture)

```
app/
├── src/main/
│   ├── java/com/bikerornot/app/
│   │   ├── core/
│   │   │   ├── di/             # Hilt modules
│   │   │   ├── navigation/     # NavGraph, Routes, NavController
│   │   │   ├── network/        # Supabase client setup
│   │   │   ├── util/           # Extensions, helpers, constants
│   │   │   └── ui/             # Theme, Typography, Colors, shared Composables
│   │   ├── data/
│   │   │   ├── remote/         # Supabase data sources
│   │   │   ├── local/          # DataStore, Room (if used)
│   │   │   ├── repository/     # Repository implementations
│   │   │   └── model/          # Data classes (DTOs)
│   │   ├── domain/
│   │   │   ├── model/          # Domain models
│   │   │   ├── repository/     # Repository interfaces
│   │   │   └── usecase/        # Use cases (one per operation)
│   │   └── presentation/
│   │       ├── auth/           # Login, Signup, Verify, Reset
│   │       ├── onboarding/     # Username, photo, bikes
│   │       ├── feed/           # Home feed, post composer
│   │       ├── profile/        # Profile, wall, photos, friends
│   │       ├── messages/       # DM list + conversation thread
│   │       ├── notifications/  # Notification list
│   │       ├── search/         # User search
│   │       └── settings/       # Account, privacy, blocked
│   └── res/
```

```
│       ├── drawable/
│       ├── values/          # strings.xml, colors.xml
│       └── xml/             # file_provider_paths.xml, network_security_config.xml
├── build.gradle.kts
└── AndroidManifest.xml
```

Each feature module under `presentation/` follows:

```
feature/
├── FeatureScreen.kt        # Composable screen
├── FeatureViewModel.kt     # ViewModel with StateFlow
├── FeatureUiState.kt       # Sealed class / data class for UI state
└── components/             # Feature-specific composables
```

Supabase Kotlin SDK Setup (Reference for All Phases)

```kotlin
// core/network/SupabaseClient.kt
val supabase = createSupabaseClient(
    supabaseUrl = BuildConfig.SUPABASE_URL,
    supabaseKey = BuildConfig.SUPABASE_ANON_KEY
) {
    install(Auth)
    install(Postgrest)
    install(Storage)
    install(Realtime)
}
```

```kotlin
// core/di/SupabaseModule.kt
@Module
@InstallIn(SingletonComponent::class)
object SupabaseModule {
    @Provides @Singleton
    fun provideSupabaseClient(): SupabaseClient = supabase
}
```

Store keys in `local.properties` (never committed to Git):

```
SUPABASE_URL=https://yourproject.supabase.co
SUPABASE_ANON_KEY=your-anon-key
```

Expose via `buildConfigField` in `build.gradle.kts`.

Standard UiState Pattern (Used in Every ViewModel)

```kotlin
// Generic sealed UiState for screens
sealed class UiState<out T> {
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error(val message: String) : UiState<Nothing>()
}

// Example ViewModel pattern
@HiltViewModel
class FeedViewModel @Inject constructor(
    private val getPostsUseCase: GetPostsUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow<UiState<List<Post>>>(UiState.Loading)
    val uiState: StateFlow<UiState<List<Post>>> = _uiState.asStateFlow()

    init { loadPosts() }

    private fun loadPosts() {
        viewModelScope.launch {
            _uiState.value = UiState.Loading
            getPostsUseCase()
                .onSuccess { _uiState.value = UiState.Success(it) }
                .onFailure { _uiState.value = UiState.Error(it.message ?: "Unknown error") }
        }
    }
}
```

---

gradle/libs.versions.toml (Version Catalog — Use for All Phases)

```toml
```

```
[versions]
kotlin = "2.0.0"
agp = "8.5.0"
compose-bom = "2024.09.00"
hilt = "2.51.1"
supabase = "3.0.0"
ktor = "3.0.0"
coil = "3.0.0"
navigation-compose = "2.8.0"
lifecycle = "2.8.5"
coroutines = "1.9.0"
datastore = "1.1.1"
room = "2.6.1"
firebase-bom = "33.4.0"

[libraries]
# Compose
compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "compose-bom" }
compose-ui = { group = "androidx.compose.ui", name = "ui" }
compose-material3 = { group = "androidx.compose.material3", name = "material3" }
compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling-preview" }

# Hilt
hilt-android = { group = "com.google.dagger", name = "hilt-android", version.ref = "hilt" }
hilt-compiler = { group = "com.google.dagger", name = "hilt-compiler", version.ref = "hilt" }
hilt-navigation-compose = { group = "androidx.hilt", name = "hilt-navigation-compose", version = "1.2.0" }

# Supabase
supabase-auth = { group = "io.github.jan-tennert.supabase", name = "auth-kt", version.ref = "supabase" }
supabase-postgrest = { group = "io.github.jan-tennert.supabase", name = "postgrest-kt", version.ref = "supabase" }
supabase-storage = { group = "io.github.jan-tennert.supabase", name = "storage-kt", version.ref = "supabase" }
supabase-realtime = { group = "io.github.jan-tennert.supabase", name = "realtime-kt", version.ref = "supabase" }
ktor-android = { group = "io.ktor", name = "ktor-client-android", version.ref = "ktor" }

# Navigation
navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigation-compose" }

# Lifecycle / ViewModel
lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-compose", version.ref = "lifecy
lifecycle-runtime-compose = { group = "androidx.lifecycle", name = "lifecycle-runtime-compose", version.ref = "lifecycle" }

# Coil
coil-compose = { group = "io.coil-kt.coil3", name = "coil-compose", version.ref = "coil" }
```

```
coil-network = { group = "io.coil-kt.coil3", name = "coil-network-ktor", version.ref = "coil" }

# Coroutines
coroutines-android = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-android", version.ref = "coroutines" }

# DataStore
datastore-preferences = { group = "androidx.datastore", name = "datastore-preferences", version.ref = "datastore" }

# Room
room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room" }
room-ktx = { group = "androidx.room", name = "room-ktx", version.ref = "room" }
room-compiler = { group = "androidx.room", name = "room-compiler", version.ref = "room" }

# Firebase
firebase-bom = { group = "com.google.firebase", name = "firebase-bom", version.ref = "firebase-bom" }
firebase-messaging = { group = "com.google.firebase", name = "firebase-messaging-ktx" }

# Testing
junit = { group = "junit", name = "junit", version = "4.13.2" }
mockk = { group = "io.mockk", name = "mockk", version = "1.13.12" }
turbine = { group = "app.cash.turbine", name = "turbine", version = "1.1.0" }
coroutines-test = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-test", version.ref = "coroutines" }
```

PHASE 1 — Project Setup & Architecture

Goal: Create a runnable Android project with all dependencies configured, Supabase connected, and the navigation skeleton in place.

Deliverables

- New Android project with Kotlin DSL Gradle + version catalog
- All dependencies from `libs.versions.toml` above added and synced
- Hilt set up with `@HiltAndroidApp` on Application class
- `SupabaseModule.kt` with DI-provided client
- `BuildConfig` fields for Supabase URL + key (from `local.properties`)
- `AppNavGraph.kt` with all routes defined (even if screens are empty stubs)
- Material 3 theme (`BikerOrNotTheme`) with custom colors:

    - Primary: Deep orange/amber (motorcycle aesthetic)
    - Background: Dark surface option for dark mode
    - Support both light and dark theme from day one
- `MainActivity.kt` hosting the NavHost
- `NetworkSecurityConfig.xml` configured
- Empty stub Composables for every screen listed in the nav graph
- Project builds and runs showing a blank nav host

App Navigation Routes

kotlin

```kotlin
sealed class Screen(val route: String) {
    // Auth
    object Login : Screen("login")
    object Signup : Screen("signup")
    object VerifyEmail : Screen("verify_email")
    object ForgotPassword : Screen("forgot_password")
    object ResetPassword : Screen("reset_password/{token}") {
        fun createRoute(token: String) = "reset_password/$token"
    }
    // Onboarding
    object Onboarding : Screen("onboarding")
    // Main
    object Feed : Screen("feed")
    object Profile : Screen("profile/{username}") {
        fun createRoute(username: String) = "profile/$username"
    }
    object Messages : Screen("messages")
    object Conversation : Screen("conversation/{id}") {
        fun createRoute(id: String) = "conversation/$id"
    }
    object Notifications : Screen("notifications")
    object Search : Screen("search")
    // Settings
    object Settings : Screen("settings")
    object SettingsPrivacy : Screen("settings/privacy")
    object SettingsNotifications : Screen("settings/notifications")
    object SettingsBlocked : Screen("settings/blocked")
}
```

Bottom Navigation Bar

Show on: Feed, Search, Messages, Notifications, Profile (own)
Hide on: Auth screens, Onboarding, Conversation detail, Settings

Startup Auth Check

```kotlin
// MainActivity — determine start destination based on session
val startDestination = when {
    !supabase.auth.currentSessionOrNull() != null -> Screen.Feed.route
    else -> Screen.Login.route
}
```

Claude Code Prompt for Phase 1

> *"I'm building the BikerOrNot Android app. Here is my full build plan: [paste this document]. Let's start with Phase 1: Project Setup & Architecture. Please create a new Android project using Kotlin, Jetpack Compose, and the Gradle version catalog defined in this document. Set up Hilt, the Supabase client module, Material 3 theming, the full navigation graph with stub screens, and the startup auth check in MainActivity. Use the project structure defined in this document."*

PHASE 2 — Authentication

Goal: Full auth flow — signup with all required fields, email verification, login, and password reset — all backed by Supabase Auth with Resend delivering emails.

Screens

- `SignupScreen` — full registration form
- `LoginScreen` — email + password
- `VerifyEmailScreen` — "Check your inbox" confirmation screen + resend option
- `ForgotPasswordScreen` — enter email to receive reset link
- `ResetPasswordScreen` — new password form (deep link from email)

Signup Form Fields

All fields required:

- First name (max 50 chars)
- Last name (max 50 chars)
- Email address
- Password (min 8 chars, 1 uppercase, 1 number, 1 special char — validate live)
- Confirm password (live match validation)
- Date of birth (DatePickerDialog — must be 18+)
- Zip code (US 5-digit or international)
- Relationship status (RadioGroup):
  - Single
  - In a Relationship
  - It's Complicated

Supabase Auth Implementation

```kotlin
// Pass all profile fields as user metadata so the DB trigger can read them
supabase.auth.signUpWith(Email) {
    email = formState.email
    password = formState.password
    data = buildJsonObject {
        put("first_name", formState.firstName)
        put("last_name", formState.lastName)
        put("date_of_birth", formState.dateOfBirth.toString()) // ISO 8601
        put("zip_code", formState.zipCode)
        put("relationship_status", formState.relationshipStatus)
    }
}
```

```kotlin
```

```
// Login
supabase.auth.signInWith(Email) {
    email = formState.email
    password = formState.password
}

// Password reset request
supabase.auth.resetPasswordForEmail(email)

// Sign out
supabase.auth.signOut()
```

Deep Link Handling for Email Verification & Password Reset

Supabase Auth sends links in format: `io.bikerornot://auth/callback`

```xml
<!-- AndroidManifest.xml — inside MainActivity intent-filter -->
<intent-filter android:autoVerify="true">
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="io.bikerornot" android:host="auth" />
</intent-filter>
```

```kotlin
// MainActivity — handle deep link on resume
override fun onNewIntent(intent: Intent) {
    super.onNewIntent(intent)
    // Pass URI to Supabase Auth for token exchange
    intent.data?.let { uri ->
        viewModelScope.launch {
            supabase.auth.parseFragmentAndImportSession(uri.toString())
        }
    }
}
```

Session Persistence

Supabase-kt handles session persistence automatically via `SessionManager`. Configure `serializer` with DataStore for more control:

```kotlin
install(Auth) {
    scheme = "io.bikerornot"
    host = "auth"
}
```

Form Validation Rules

- Real-time validation (validate on value change, show error after first touch)
- Submit button disabled until all fields valid
- Show password strength indicator on password field
- Age check: DOB must be at least 18 years before today

UiState for Auth

```kotlin
data class AuthUiState(
    val isLoading: Boolean = false,
    val error: String? = null,
    val isSuccess: Boolean = false
)

// Form state (separate from UiState)
data class SignupFormState(
    val firstName: String = "",
    val lastName: String = "",
    val email: String = "",
    val password: String = "",
    val confirmPassword: String = "",
    val dateOfBirth: LocalDate? = null,
    val zipCode: String = "",
    val relationshipStatus: String = "",
    // Validation errors
    val emailError: String? = null,
    val passwordError: String? = null,
    val confirmPasswordError: String? = null,
    val dateOfBirthError: String? = null
)
```

Claude Code Prompt for Phase 2

> *"Continuing BikerOrNot Android build — Phase 2: Authentication. Reference the full build plan document. Build the complete auth flow: SignupScreen with all 8 fields (including DOB date picker with 18+ validation, relationship status radio group), LoginScreen, VerifyEmailScreen, ForgotPasswordScreen, and ResetPasswordScreen. Implement deep link handling in MainActivity for Supabase Auth callbacks. Use the MVVM pattern with Hilt-injected ViewModels and StateFlow UiState as defined in the document."*

PHASE 3 — Onboarding & Profile

Goal: First-time user onboarding flow (username + photo + bike), profile page with tabs, and profile editing.

Onboarding Flow (shown once after email verification)

Paged flow using `HorizontalPager` (Accompanist or built-in Compose):

- **Page 1 — Pick a Username:** Text field, real-time availability check via Supabase RPC, alphanumeric + underscores, 3–30 chars
- **Page 2 — Profile Photo:** Camera or gallery picker, circular crop preview (use `uCrop` library or custom Compose crop), upload to Supabase Storage `avatars/` bucket
- **Page 3 — Your Bikes:** Add up to 5 bikes (year + make + model), skip option available
- **Finish:** Mark `onboarding_complete = true` on profile, navigate to Feed

Profile Photo Upload

```kotlin
```

```kotlin
// Compress before upload
val compressed = Compressor.compress(context, imageFile) {
    resolution(800, 800)
    quality(80)
    format(Bitmap.CompressFormat.JPEG)
    size(500_000) // 500KB max
}

// Upload to Supabase Storage
val path = "avatars/${userId}.jpg"
supabase.storage.from("avatars").upload(
    path = path,
    data = compressed.readBytes(),
    upsert = true
)

// Save path to profile (not full URL — generate URLs in UI)
updateProfile(profilePhotoUrl = path)
```

Supabase Storage URL with Transformation

```kotlin
// Utility function — use everywhere, don't build URLs manually
fun getAvatarUrl(path: String, size: Int = 80): String {
    return supabase.storage.from("avatars")
        .publicUrl(path) {
            transform {
                width = size
                height = size
                resize = ImageTransformation.Resize.COVER
            }
        }
}
```

Profile Screen Layout

```
┌──────────────────────────────────────┐
│        Cover Photo                   │ ← Tap to change (own profile)
│   [Avatar]  Name  @username          │
│   Bio · Location · Riding            │
│   Relationship status badge          │
```

```
│ [Add Friend] [Message] [Block] │ ← Hidden on own profile
├──────────────────────────────────────┤
│ Wall │ Photos │ Friends │ About │ ← TabRow
├──────────────────────────────────────┤
│      Tab Content          │
└──────────────────────────────────────┘
```

Profile Data Model

kotlin

```kotlin
@Serializable
data class Profile(
    val id: String,
    val username: String,
    @SerialName("first_name") val firstName: String,
    @SerialName("last_name") val lastName: String,
    @SerialName("date_of_birth") val dateOfBirth: String,
    @SerialName("zip_code") val zipCode: String,
    @SerialName("relationship_status") val relationshipStatus: String,
    @SerialName("display_name") val displayName: String? = null,
    val bio: String? = null,
    val location: String? = null,
    @SerialName("riding_style") val ridingStyle: List<String> = emptyList(),
    @SerialName("profile_photo_url") val profilePhotoUrl: String? = null,
    @SerialName("cover_photo_url") val coverPhotoUrl: String? = null,
    val status: String = "active",
    val role: String = "user",
    @SerialName("onboarding_complete") val onboardingComplete: Boolean = false,
    @SerialName("created_at") val createdAt: String
)
```

Coil Image Loading

kotlin

```
// Avatar component — reuse everywhere
@Composable
fun UserAvatar(
    profilePath: String?,
    size: Dp = 40.dp,
    modifier: Modifier = Modifier
) {
    AsyncImage(
        model = profilePath?.let { getAvatarUrl(it, size.value.toInt()) },
        contentDescription = "Avatar",
        contentScale = ContentScale.Crop,
        placeholder = painterResource(R.drawable.ic_avatar_placeholder),
        error = painterResource(R.drawable.ic_avatar_placeholder),
        modifier = modifier
            .size(size)
            .clip(CircleShape)
    )
}
```

Settings Screen

- Edit profile (all fields)
- Change username (once per 30 days — enforce via timestamp check)
- Change email → triggers re-verification
- Change password
- Privacy: profile visibility (public / friends only)
- Notification preferences
- Blocked users
- Sign out
- Delete account (confirmation dialog, then soft delete)

Claude Code Prompt for Phase 3

> *"Phase 3: Onboarding & Profile. Build the 3-page HorizontalPager onboarding flow (username availability check, profile photo upload with compression to Supabase Storage, bike entry). Build the profile screen with TabRow (Wall/Photos/Friends/About tabs), the UserAvatar composable, and Supabase Storage transformation URL utility. Build the Settings screen with all edit fields. Use the Profile data model defined in the document."*

PHASE 4 — Feed & Post Composer

Goal: Home feed showing friends' posts, personal wall, post composer with multi-image upload, and image gallery display.

Feed Architecture

- `LazyColumn` with `items(posts, key = { it.id })`
- Cursor-based pagination: use `created_at` of last loaded post as cursor
- Load 10 posts per page
- Pull-to-refresh with `rememberPullToRefreshState()`
- "New posts available" banner (from Realtime Broadcast — Phase 5)

Post Data Models

```kotlin
@Serializable
data class Post(
    val id: String,
    @SerialName("author_id") val authorId: String,
    @SerialName("wall_owner_id") val wallOwnerId: String? = null,
    val content: String?,
    val author: Profile? = null,          // joined
    val images: List<PostImage> = emptyList(), // joined
    @SerialName("like_count") val likeCount: Int = 0,
    @SerialName("comment_count") val commentCount: Int = 0,
    @SerialName("is_liked_by_me") val isLikedByMe: Boolean = false,
    @SerialName("created_at") val createdAt: String,
    @SerialName("deleted_at") val deletedAt: String? = null
)

@Serializable
data class PostImage(
    val id: String,
    @SerialName("post_id") val postId: String,
    @SerialName("storage_path") val storagePath: String,
    @SerialName("order_index") val orderIndex: Int
)
```

Post Composer

- Floating action button on feed → opens bottom sheet composer
- `OutlinedTextField` for text (max 2,000 chars, character counter)
- Image picker row: tap "+" → opens photo picker (`ActivityResultContracts.PickMultipleVisualMedia`, max 10)
- Show selected image thumbnails in horizontal `LazyRow` with remove buttons
- On submit:
  1. Compress each image (`Compressor` → target 500KB, 1200px max dimension)
  2. Upload to Supabase Storage `posts/` bucket in parallel (use `async { }` + `awaitAll()`)
  3. Insert post record with `storage_path` list
  4. Optimistic insert into local list before API response

Post Image Gallery Composable

```kotlin
// Adaptive grid: 1 image = full width, 2 = side by side, 3-4 = 2x2 grid, 5+ = first large + grid
@Composable
fun PostImageGallery(
    images: List<PostImage>,
    onImageClick: (Int) -> Unit // index → open fullscreen lightbox
)
```

Fullscreen Image Viewer

- `HorizontalPager` with zoom support (`rememberTransformableState`)
- Swipe down to dismiss
- Show image count (1/3)
- Share and download options in top action bar

Post Card Composable

```kotlin
```

```kotlin
@Composable
fun PostCard(
    post: Post,
    currentUserId: String,
    onLike: (String) -> Unit,
    onComment: (String) -> Unit,
    onProfileClick: (String) -> Unit,
    onMenuClick: (String) -> Unit   // edit / delete / report
)
```

Feed ViewModel

```kotlin
@HiltViewModel
class FeedViewModel @Inject constructor(
    private val postRepository: PostRepository
) : ViewModel() {
    private val _posts = MutableStateFlow<List<Post>>(emptyList())
    val posts: StateFlow<List<Post>> = _posts.asStateFlow()

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading.asStateFlow()

    private val _hasNewPosts = MutableStateFlow(false)
    val hasNewPosts: StateFlow<Boolean> = _hasNewPosts.asStateFlow()

    private var cursor: String? = null
    private var hasMore = true

    fun loadNextPage() { /* cursor pagination */ }
    fun refresh() { cursor = null; hasMore = true; loadNextPage() }
    fun createPost(content: String, images: List<Uri>) { /* compress + upload + insert */ }
    fun likePost(postId: String) { /* optimistic toggle */ }
    fun deletePost(postId: String) { /* optimistic remove */ }
}
```

Claude Code Prompt for Phase 4

> *"Phase 4: Feed & Post Composer. Build the home feed with cursor-based pagination in a LazyColumn with pull-to-refresh. Build the post composer as a ModalBottomSheet with multi-image picker (max 10), image compression, parallel Supabase Storage upload, and optimistic insert. Build the PostCard composable with*

> *like/comment buttons, the PostImageGallery adaptive grid, and a fullscreen HorizontalPager image viewer. Use the data models and ViewModel pattern from the build plan."*

PHASE 5 — Comments, Likes & Realtime

Goal: Full comment system with real-time updates via Supabase Realtime, like functionality on posts and comments.

Comments UI

- Tap comment count on PostCard → expands inline comment section (no separate screen)
- `LazyColumn` nested inside post expansion (use `nestedScroll` carefully — or a separate detail screen for posts with many comments)
- Show top 3 comments inline, "View all X comments" button loads rest
- Text input + send button pinned at bottom of comment section
- Real-time: new comments appear instantly while section is open

Supabase Realtime for Comments

```kotlin
```

```kotlin
// In CommentsViewModel — subscribe when screen opens, unsubscribe on clear
private var realtimeChannel: RealtimeChannel? = null

fun subscribeToComments(postId: String) {
    realtimeChannel = supabase.realtime.channel("comments:$postId")

    realtimeChannel?.postgresChangeFlow<PostgresAction.Insert>(schema = "public") {
        table = "comments"
        filter = "post_id=eq.$postId"
    }?.onEach { change ->
        val newComment = change.decodeRecord<Comment>()
        _comments.update { current -> current + newComment }
    }?.launchIn(viewModelScope)

    viewModelScope.launch {
        realtimeChannel?.subscribe()
    }
}

override fun onCleared() {
    viewModelScope.launch {
        realtimeChannel?.unsubscribe()
    }
}
```

Supabase Realtime for Feed (Broadcast)

```kotlin
// FeedViewModel — listen for new posts from friends (Broadcast, not Postgres Changes)
fun subscribeFeedBroadcast(userId: String) {
    val channel = supabase.realtime.channel("feed:$userId")

    channel.broadcastFlow<JsonObject>(event = "new_post")
        .onEach { _hasNewPosts.value = true }
        .launchIn(viewModelScope)

    viewModelScope.launch { channel.subscribe() }
}
// When user taps "New posts available" banner → refresh() + _hasNewPosts.value = false
```

Comment Data Model

```kotlin
@Serializable
data class Comment(
    val id: String,
    @SerialName("post_id") val postId: String,
    @SerialName("author_id") val authorId: String,
    val content: String,
    val author: Profile? = null,
    @SerialName("like_count") val likeCount: Int = 0,
    @SerialName("is_liked_by_me") val isLikedByMe: Boolean = false,
    @SerialName("created_at") val createdAt: String
)
```

Optimistic Like Toggle

```kotlin
fun togglePostLike(postId: String) {
    // 1. Immediately update local state
    _posts.update { posts ->
        posts.map { post ->
            if (post.id == postId) {
                post.copy(
                    isLikedByMe = !post.isLikedByMe,
                    likeCount = if (post.isLikedByMe) post.likeCount - 1 else post.likeCount + 1
                )
            } else post
        }
    }
    // 2. Send to Supabase (rollback on error)
    viewModelScope.launch {
        postRepository.toggleLike(postId)
            .onFailure { /* rollback optimistic update */ }
    }
}
```

Claude Code Prompt for Phase 5

> *"Phase 5: Comments, Likes & Realtime. Build the expandable inline comment section on PostCard with a LazyColumn of CommentItems, text input, and send button. Implement Supabase Realtime Postgres*

> *Changes subscription in CommentsViewModel filtered by post_id. Implement optimistic like toggling for posts and comments. Set up the Realtime Broadcast subscription in FeedViewModel for new post notifications with the 'New posts available' banner. Use the Realtime patterns defined in the build plan."*

PHASE 6 — Friend Requests & Friends List

Goal: Send, accept, decline, and cancel friend requests. View friends list on profiles. Mutual friend count in search results.

Friendship States for Profile Action Button

```kotlin
enum class FriendshipStatus {
    NONE,           // Show "Add Friend" button
    PENDING_SENT,   // Show "Cancel Request" button
    PENDING_RECEIVED, // Show "Accept" + "Decline" buttons
    FRIENDS         // Show "Friends ▾" dropdown (Unfriend option)
}
```

Profile Action Bar

```kotlin
@Composable
fun ProfileActionBar(
    friendshipStatus: FriendshipStatus,
    onAddFriend: () -> Unit,
    onCancelRequest: () -> Unit,
    onAcceptRequest: () -> Unit,
    onDeclineRequest: () -> Unit,
    onUnfriend: () -> Unit,
    onMessage: () -> Unit,
    onBlock: () -> Unit
)
```

Supabase Queries

```kotlin
```

```
// Get friendship status between current user and target
suspend fun getFriendshipStatus(targetId: String): FriendshipStatus {
    val result = supabase.postgrest.rpc("get_friendship_status", buildJsonObject {
        put("user_a", currentUserId)
        put("user_b", targetId)
    })
    // Map DB status string to FriendshipStatus enum
}

// Send friend request
suspend fun sendFriendRequest(addresseeId: String) {
    supabase.postgrest["friendships"].insert(buildJsonObject {
        put("requester_id", currentUserId)
        put("addressee_id", addresseeId)
    })
}

// Accept friend request
suspend fun acceptFriendRequest(requesterId: String) {
    supabase.postgrest["friendships"]
        .update({ set("status", "accepted") }) {
            filter { eq("requester_id", requesterId); eq("addressee_id", currentUserId) }
        }
}

// Delete (decline / cancel / unfriend)
suspend fun deleteFriendship(otherUserId: String) {
    supabase.postgrest["friendships"].delete {
        filter {
            or {
                and { eq("requester_id", currentUserId); eq("addressee_id", otherUserId) }
                and { eq("requester_id", otherUserId); eq("addressee_id", currentUserId) }
            }
        }
    }
}
```

Friends Tab on Profile

- `LazyVerticalGrid(columns = GridCells.Fixed(3))` showing friend avatars + names
- Tap friend → navigate to their profile
- Empty state: "No friends yet" with an Add Friends suggestion

Friend Requests Screen

- Accessible from notification bell or dedicated screen
- Two sections: "Received" (accept/decline) + "Sent" (cancel)
- Real-time: subscribe to `friendships` Postgres Changes filtered by `addressee_id` = current user

Claude Code Prompt for Phase 6

> *"Phase 6: Friend Requests & Friends List. Build the FriendshipStatus enum and ProfileActionBar composable with state-based buttons. Implement all friendship Supabase queries (send, accept, decline, cancel, unfriend) in a FriendshipRepository. Build the Friends tab on the profile as a LazyVerticalGrid. Build a FriendRequestsScreen showing received and sent requests with real-time Supabase Realtime subscription. Use Hilt for DI as established in the project."*

PHASE 7 — Blocking & Search

Goal: Block/unblock users, enforce block rules platform-wide, and full-text user search.

Blocking

Block Action

- Available from: profile page "⋯" menu, post card "⋯" menu, message thread
- On block: confirm via `AlertDialog`, then:
  1. Insert into `blocks` table
  2. Delete existing friendship if present (call `deleteFriendship()` from Phase 6)
  3. Navigate away from their profile / remove from feed

```kotlin
```

```
suspend fun blockUser(blockedId: String) {
    supabase.postgrest["blocks"].insert(buildJsonObject {
        put("blocker_id", currentUserId)
        put("blocked_id", blockedId)
    })
    // RLS on the DB ensures blocked content is filtered at query level
    // Locally remove blocked user's posts from cached feed
    _posts.update { posts -> posts.filter { it.authorId != blockedId } }
}

suspend fun unblockUser(blockedId: String) {
    supabase.postgrest["blocks"].delete {
        filter { eq("blocker_id", currentUserId); eq("blocked_id", blockedId) }
    }
}

suspend fun getBlockedUsers(): List<Profile> {
    return supabase.postgrest["blocks"]
        .select("blocked_id, profiles!blocked_id(*)") {
            filter { eq("blocker_id", currentUserId) }
        }
        .decodeList<BlockWithProfile>()
        .map { it.profile }
}
```

Settings → Blocked Users Screen

- `LazyColumn` of blocked users with Unblock button per row
- Empty state with explanation text

Search

Search Screen

```
┌──────────────────────────────────────┐
│ 🔍 Search bikers...                  │ ← SearchBar (always focused on enter)
├──────────────────────────────────────┤
│ [Avatar] John Doe  @johndoe          │
│          2 mutual friends            │
│ [Add Friend]  [Message]              │
├──────────────────────────────────────┤
│ [Avatar] Jane Smith @janesmith       │
```

```
│ ...                    │
└──────────────────────────────────────────────┘
```

Debounced Search with Supabase RPC

```kotlin
// SearchViewModel
private val searchQuery = MutableStateFlow("")

init {
    searchQuery
        .debounce(300)
        .filter { it.length >= 2 }
        .distinctUntilChanged()
        .flatMapLatest { query ->
            flow {
                emit(UiState.Loading)
                searchRepository.searchUsers(query)
                    .onSuccess { emit(UiState.Success(it)) }
                    .onFailure { emit(UiState.Error(it.message ?: "Search failed")) }
            }
        }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), UiState.Loading)
}

fun onQueryChange(query: String) { searchQuery.value = query }
```

```kotlin
// Call the Postgres FTS RPC function defined in the DB schema
suspend fun searchUsers(query: String): Result<List<Profile>> {
    return runCatching {
        supabase.postgrest.rpc("search_users", buildJsonObject {
            put("search_term", query)
            put("current_user_id", currentUserId)
        }).decodeList<Profile>()
    }
}
```

Recent Searches

- Store last 5 search terms in DataStore

- Show as chips when search bar is empty

Claude Code Prompt for Phase 7

> *"Phase 7: Blocking & Search. Implement block/unblock in BlockRepository with Supabase queries. Build the Blocked Users settings screen. Build the SearchScreen with SearchBar, debounced Flow query (300ms, min 2 chars), Supabase RPC call to search_users, result items showing avatar/name/mutual friends/action button. Store recent searches in DataStore and show as chips when query is empty. Enforce block rules by filtering cached feed posts on block action."*

PHASE 8 — Direct Messaging

Goal: Full real-time DM system between friends. Conversation list, message thread, seen receipts, unread badge.

Screens

- `MessagesScreen` — list of conversations, sorted by most recent message
- `ConversationScreen` — message thread for one conversation

Data Models

```kotlin
```

```kotlin
@Serializable
data class Conversation(
    val id: String,
    val participants: List<Profile>,
    @SerialName("last_message") val lastMessage: Message? = null,
    @SerialName("unread_count") val unreadCount: Int = 0,
    @SerialName("created_at") val createdAt: String
)

@Serializable
data class Message(
    val id: String,
    @SerialName("conversation_id") val conversationId: String,
    @SerialName("sender_id") val senderId: String,
    val content: String,
    val sender: Profile? = null,
    @SerialName("created_at") val createdAt: String,
    @SerialName("deleted_at") val deletedAt: String? = null
)
```

Conversation Screen Layout

```kotlin
```

```kotlin
// Standard messaging layout
Scaffold(
    topBar = { ConversationTopBar(participant) },
    bottomBar = {
        MessageInputBar(
            value = inputText,
            onValueChange = { inputText = it },
            onSend = { viewModel.sendMessage(it) }
        )
    }
) { padding ->
    LazyColumn(
        reverseLayout = true,   // newest at bottom
        state = listState,
        modifier = Modifier.padding(padding)
    ) {
        items(messages, key = { it.id }) { message ->
            MessageBubble(message, isFromMe = message.senderId == currentUserId)
        }
    }
}
```

Real-Time Message Subscription

```kotlin
// ConversationViewModel
fun subscribeToMessages(conversationId: String) {
    val channel = supabase.realtime.channel("messages:$conversationId")

    channel.postgresChangeFlow<PostgresAction.Insert>(schema = "public") {
        table = "messages"
        filter = "conversation_id=eq.$conversationId"
    }.onEach { action ->
        val newMessage = action.decodeRecord<Message>()
        _messages.update { it + newMessage }
        markAsRead(conversationId)
    }.launchIn(viewModelScope)

    viewModelScope.launch { channel.subscribe() }
}
```

Unread Count Badge (Nav Bar)

```kotlin
// MessagesViewModel — subscribe to all conversations for unread count
// Use Postgres Changes on conversation_participants filtered by user_id
// Badge count = sum of unread_count across all conversations
// Update last_read_at when user opens a conversation
```

Send Message

```kotlin
suspend fun sendMessage(conversationId: String, content: String) {
    // Optimistic insert
    val tempMessage = Message(id = UUID.randomUUID().toString(), ...)
    _messages.update { it + tempMessage }

    supabase.postgrest["messages"].insert(buildJsonObject {
        put("conversation_id", conversationId)
        put("sender_id", currentUserId)
        put("content", content)
    })
}
```

Message Bubble

- Own messages: right-aligned, primary color background
- Others' messages: left-aligned, surface color background
- Show avatar only for others (not repeated per message — show only on first in a group)
- Timestamp: show only on last message in a group within 1 minute
- Deleted message: show "This message was deleted" in italic, greyed out
- Long press → delete own message option (`DropdownMenu`)

Create New Conversation

- From profile page "Message" button
- Check if conversation already exists → navigate to it, or create new one

```kotlin
```

```kotlin
suspend fun getOrCreateConversation(otherUserId: String): String {
    // Query existing conversation with both participants
    // If not found → create conversation + insert both participants
    // Return conversation ID
}
```

Claude Code Prompt for Phase 8

> *"Phase 8: Direct Messaging. Build MessagesScreen (conversation list with unread badges, sorted by most recent message). Build ConversationScreen with reverseLayout LazyColumn, MessageBubble composable (own/other styling, deleted state, grouping logic), MessageInputBar, and real-time Supabase Realtime Postgres Changes subscription. Implement getOrCreateConversation for the profile Message button. Implement unread badge count via Realtime subscription in MessagesViewModel. Use the data models and patterns from the build plan."*

PHASE 9 — Notifications

Goal: In-app notification list with real-time badge updates AND push notifications via Firebase Cloud Messaging.

In-App Notifications

Notification Data Model

```kotlin
```

```kotlin
@Serializable
data class AppNotification(
    val id: String,
    @SerialName("recipient_id") val recipientId: String,
    @SerialName("actor_id") val actorId: String,
    val type: NotificationType,
    @SerialName("entity_id") val entityId: String? = null,
    @SerialName("entity_type") val entityType: String? = null,
    val actor: Profile? = null,
    val read: Boolean = false,
    @SerialName("created_at") val createdAt: String
)

enum class NotificationType {
    FRIEND_REQUEST,
    FRIEND_ACCEPT,
    POST_LIKE,
    COMMENT,
    WALL_POST,
    MESSAGE
}
```

Realtime Badge Update

```kotlin
// NotificationsViewModel — subscribe on app start
fun subscribeToNotifications(userId: String) {
    val channel = supabase.realtime.channel("notifications:$userId")

    channel.postgresChangeFlow<PostgresAction.Insert>(schema = "public") {
        table = "notifications"
        filter = "recipient_id=eq.$userId"
    }.onEach { action ->
        val notification = action.decodeRecord<AppNotification>()
        _notifications.update { listOf(notification) + it }
        _unreadCount.update { it + 1 }
    }.launchIn(viewModelScope)

    viewModelScope.launch { channel.subscribe() }
}
```

Notification Item Tap Behavior

Type	Navigate to
FRIEND_REQUEST	Friend requests screen
FRIEND_ACCEPT	Accepted user's profile
POST_LIKE	Post (expand from feed or detail screen)
COMMENT	Post with comment section expanded
WALL_POST	Own profile wall
MESSAGE	Conversation thread

Mark as Read

- Mark individual on tap
- "Mark all as read" button in top bar
- Unread items shown with a colored left border or dot indicator

Push Notifications (Firebase Cloud Messaging)

Setup

1. Add `google-services.json` to `app/` directory
2. Apply `com.google.gms.google-services` plugin in Gradle
3. Add `firebase-messaging-ktx` dependency

FCM Service

```kotlin
```

```kotlin
@AndroidEntryPoint
class BikerOrNotFirebaseService : FirebaseMessagingService() {

    @Inject lateinit var notificationHelper: NotificationHelper

    override fun onMessageReceived(message: RemoteMessage) {
        val title = message.notification?.title ?: return
        val body = message.notification?.body ?: return
        val data = message.data

        notificationHelper.showNotification(
            title = title,
            body = body,
            deepLinkRoute = data["route"] // e.g. "conversation/abc123"
        )
    }

    override fun onNewToken(token: String) {
        // Save new FCM token to Supabase user profile
        viewModelScope.launch {
            profileRepository.updateFcmToken(token)
        }
    }
}
```

Notification Helper

```kotlin
```

```kotlin
class NotificationHelper @Inject constructor(@ApplicationContext private val context: Context) {

    fun showNotification(title: String, body: String, deepLinkRoute: String?) {
        val pendingIntent = deepLinkRoute?.let { route ->
            NavDeepLinkBuilder(context)
                .setGraph(R.navigation.nav_graph)
                .setDestination(/* map route to destination */)
                .createPendingIntent()
        }

        val notification = NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_notification)
            .setContentTitle(title)
            .setContentText(body)
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true)
            .setContentIntent(pendingIntent)
            .build()

        NotificationManagerCompat.from(context).notify(Random.nextInt(), notification)
    }
}
```

FCM Token → Supabase

Add `fcm_token text` column to `profiles` table. Update on app start + `onNewToken`.

Notification Channel Setup (required for Android 8+)

```kotlin
// In Application.onCreate()
val channel = NotificationChannel(
    "bikerornot_default",
    "BikerOrNot Notifications",
    NotificationManager.IMPORTANCE_DEFAULT
)
getSystemService(NotificationManager::class.java).createNotificationChannel(channel)
```

Request Notification Permission (Android 13+)

```kotlin
```

```kotlin
// In MainActivity or Onboarding
val permissionLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()
) { /* handle result */ }

LaunchedEffect(Unit) {
    if (Build.VERSION.SDK_INT >= 33 &&
        ContextCompat.checkSelfPermission(context, POST_NOTIFICATIONS) != GRANTED) {
        permissionLauncher.launch(POST_NOTIFICATIONS)
    }
}
```

Claude Code Prompt for Phase 9

> *"Phase 9: Notifications. Build NotificationsScreen with a LazyColumn of AppNotificationItems (icon, text, timestamp, read indicator, tap navigation). Implement Supabase Realtime subscription for live badge updates in NotificationsViewModel. Set up Firebase Cloud Messaging: BikerOrNotFirebaseService, NotificationHelper with deep link PendingIntent, FCM token storage in Supabase, notification channel creation in Application class, and POST_NOTIFICATIONS permission request for Android 13+. Use the data models from the build plan."*

PHASE 10 — Polish, Performance & Launch Prep

Goal: Image caching strategy, error handling, loading states, offline awareness, accessibility, Play Store preparation, and final security hardening.

Image Caching (Coil)

```kotlin
```

```kotlin
// Application class — configure Coil singleton
SingletonImageLoader.setSafe(this) {
    ImageLoader.Builder(this)
        .memoryCache {
            MemoryCache.Builder()
                .maxSizePercent(this@App, 0.25) // 25% of app memory
                .build()
        }
        .diskCache {
            DiskCache.Builder()
                .directory(cacheDir.resolve("image_cache"))
                .maxSizeBytes(100L * 1024 * 1024) // 100MB disk cache
                .build()
        }
        .networkCachePolicy(CachePolicy.ENABLED)
        .diskCachePolicy(CachePolicy.ENABLED)
        .build()
}
```

Offline Awareness

```kotlin
// Network status monitor using ConnectivityManager
class NetworkMonitor @Inject constructor(@ApplicationContext context: Context) {
    val isOnline: Flow<Boolean> = callbackFlow {
        val manager = context.getSystemService(ConnectivityManager::class.java)
        val callback = object : ConnectivityManager.NetworkCallback() {
            override fun onAvailable(network: Network) { trySend(true) }
            override fun onLost(network: Network) { trySend(false) }
        }
        manager.registerDefaultNetworkCallback(callback)
        awaitClose { manager.unregisterNetworkCallback(callback) }
    }
}
// Show "You're offline" banner in Scaffold when isOnline = false
```

Global Error Handling

```kotlin
```

```kotlin
// Sealed error type for user-facing messages
sealed class AppError {
    data class Network(val message: String = "Check your connection and try again") : AppError()
    data class Auth(val message: String = "Please sign in again") : AppError()
    data class NotFound(val message: String = "This content is no longer available") : AppError()
    data class Unknown(val message: String = "Something went wrong") : AppError()
}

// Map Supabase exceptions to AppError in repository layer
fun Throwable.toAppError(): AppError = when (this) {
    is RestException -> when (statusCode) {
        401 -> AppError.Auth()
        404 -> AppError.NotFound()
        else -> AppError.Unknown(message ?: "Unknown error")
    }
    is IOException -> AppError.Network()
    else -> AppError.Unknown(message ?: "Unknown error")
}
```

Loading & Empty States

Every screen must handle all four states:

```kotlin
// Standard screen state handler
@Composable
fun <T> ScreenContent(
    uiState: UiState<T>,
    onRetry: () -> Unit,
    emptyContent: @Composable () -> Unit = { DefaultEmptyState() },
    content: @Composable (T) -> Unit
) {
    when (uiState) {
        is UiState.Loading -> LoadingSpinner()
        is UiState.Error -> ErrorState(message = uiState.message, onRetry = onRetry)
        is UiState.Success -> {
            if (uiState.data is List<*> && (uiState.data as List<*>).isEmpty()) emptyContent()
            else content(uiState.data)
        }
    }
}
```

Skeleton Loading (Shimmer)

- Install `com.valentinilk.shimmer:shimmer:1.3.0`
- Create `PostCardSkeleton` and `ProfileSkeleton` composables
- Show while initial data loads instead of spinner

Accessibility

- All interactive elements have `contentDescription`
- Font scaling supported (use `sp` for text, test at 200% font size)
- Color contrast meets WCAG AA minimum
- Touch targets minimum 48dp×48dp
- Screen reader tested with TalkBack

Security Hardening

```xml
<!-- res/xml/network_security_config.xml -->
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system" />
    </trust-anchors>
  </base-config>
</network-security-config>
```

- `BuildConfig` fields for all secrets (never hardcoded)
- `local.properties` in `.gitignore`
- Certificate pinning for Supabase domain (optional but recommended)
- ProGuard/R8 rules configured for release build — ensure Supabase + Kotlinx serialization classes not stripped

```
# proguard-rules.pro
-keep class com.bikerornot.app.data.model.** { *; }
-keep @kotlinx.serialization.Serializable class * { *; }
-keepclassmembers class kotlinx.serialization.json.** { *** Companion; }
```

Play Store Preparation

- App signing keystore created and stored securely (not in Git)
- `release` build type configured in Gradle with minification + resource shrinking
- App icons: all densities (`mdpi` through `xxxhdpi`) + adaptive icon for Android 8+
- Screenshots: phone + 7-inch tablet formats
- Short description (80 chars max) and full description drafted
- Privacy policy URL (required — create a simple one for launch)
- Content rating questionnaire completed in Play Console
- Target audience: 18+ (required — app has age gate at signup)

Pre-Launch Checklist

```
□ All Supabase free tier limits reviewed — DB < 400MB, Storage < 800MB
□ Supabase upgraded to Pro before public launch
□ Vercel upgraded to Pro before public launch (required for commercial use)
□ FCM push notifications tested on physical device
□ Deep links tested (email verification + password reset)
□ All screens tested with no internet connection
□ All screens tested with TalkBack enabled
□ ProGuard release build tested — no crashes
□ API 26 minimum SDK tested on older device/emulator
□ Play Store listing complete with screenshots and description
□ Privacy policy live at a public URL
□ Supabase RLS policies reviewed — no unauthorized data access
□ Service role key confirmed never present in APK
□ google-services.json confirmed NOT committed to Git
□ local.properties confirmed NOT committed to Git
```

Claude Code Prompt for Phase 10

> *"Phase 10: Polish, Performance & Launch Prep. Configure Coil with 100MB disk cache and 25% memory cache. Build the NetworkMonitor and offline banner. Create the ScreenContent composable for standardized loading/error/empty/success states. Build PostCardSkeleton with shimmer loading. Implement global error mapping from Supabase exceptions to AppError. Configure ProGuard rules for release builds. Set up the adaptive app icon. Review all screens for 48dp minimum touch targets and contentDescription accessibility. Walk through the pre-launch checklist."*

Shared Supabase Backend Reference

The Android app uses the same Supabase project as the web app. No changes needed to the backend. All tables, RLS policies, RPC functions, and Storage buckets are already defined in the web app's `docs/feature-outline-v3.md`.

Key RPC functions the Android app calls:

- `get_friendship_status(user_a, user_b)` — returns friendship status string
- `search_users(search_term, current_user_id)` — returns matching profiles

Storage buckets used:

- `avatars` — profile photos
- `covers` — cover/banner photos
- `posts` — post images

Realtime channels used:

- `comments:{postId}` — Postgres Changes, comment inserts
- `feed:{userId}` — Broadcast, new post signal
- `messages:{conversationId}` — Postgres Changes, message inserts
- `notifications:{userId}` — Postgres Changes, notification inserts

Quick Reference: Claude Code Session Starter

At the beginning of every Claude Code session, use this opener:

> *"I'm building the BikerOrNot Android app using Kotlin, Jetpack Compose, Hilt, Supabase (supabase-kt), and Coil. The project is in the `bikerornot` folder. Here is my full Android build plan: [paste this document]. We are currently on Phase [N]. [Describe what was completed last session if resuming]. Please [specific task for this session]."*

Android build plan — 10 phases · Kotlin · Jetpack Compose · Shared Supabase backend with web app